

三、 研究計畫內容

時變圖 (temporal graph；亦稱動態圖 dynamic graph) 是近年備受關注的研究主題。與傳統的靜態圖 (static graph) 不同，時變圖中的邊會隨時間出現或消失；換言之，每一條邊皆對應一組「可用時間點」，僅在特定時刻方可通行。此種動態連結性廣泛存在於實務系統中，例如：公共運輸系統中兩站間的列車僅於特定時刻運行 [SWP⁺14]、物流運輸網中，上游倉庫至下游配送中心的幹線運輸需依時刻表發車 [KKGB10]、以及衛星通訊網路中，衛星與地面站的通訊連結僅在預定時間內發生 [TMGV23]。此外，經貿網路 [PE14, RC13] 與生態網路 [PD05, BWD⁺12] 等連結隨時間演化的系統，亦常以時變圖表示。

由於邊集合會隨時間改變，傳統靜態圖演算法往往無法直接適用。其根本原因在於：節點 u 是否能到達節點 v ，取決於路徑上各邊的可用時間點是否滿足時間序，而非僅取決於邊是否存在。舉例而言，考慮一個物流運輸網，包含三個站點 a, b, c ，且 $a \rightarrow b$ 與 $b \rightarrow c$ 的車程皆為一小時。若 a 於 15:00 發車往 b ，且 b 於 16:00 之後仍有車發往 c ，則貨物可順利由 a 運送至 c (見圖 1 左圖)；反之，若 b 在 16:00 後無車次發往 c ，則今日無法由 a 抵達 c (見圖 1 右圖)。

本研究計畫主要探討時變圖上的網路設計 (network design) 問題：在給定服務需求與限制下，決定各物流站點於哪些時間啟用運輸 (亦即確定各邊的可用時間點)，目標是最小化發車成本 (可用時間點總數)。

此研究計畫主要受到 Carnevale 等人近期的研究啟發 [CDO25]。該研究探討如何指派最少可用時間點 (最少貨車班次)，以確保圖中任一節點皆可抵達其他節點。該問題已被證明為 NP-hard，雖然 Carnevale 等人針對不同場景提出不同近似演算法 (approximation algorithm)，但現有方法仍存在顯著的限制：

1. 產生的路徑可能過長，造成不必要的運輸延遲。
2. 未考慮貨車的運輸容量限制。

基於上述缺口，本計畫擬在 Carnevale 等人的研究框架下，進一步引入拉伸程度 (stretch) 與容量限制 (capacity) 等實務限制。由於此類擴展問題大多仍屬 NP-hard，我們將致力於設計高效的近似演算法，或針對特定圖形結構設計多項式時間的精確演算法 (exact algorithm)；同時，亦將在常見的複雜度假設下 (如 $P \neq NP$)，探討問題的近似比下界 (inapproximability)。

此計畫書的架構如下：

1. 第 (一) 小節：介紹時變圖與此計畫所探討研究問題的定義。
2. 第 (二) 小節：介紹時變圖的相關研究。
3. 第 (三) 小節：介紹 Carnevale 等人之核心演算法概念 [CDO25]，作為本計畫之基礎。
4. 第 (四) 與第 (五) 小節：討論第一年與第二年之詳細研究方法。



圖 1: 兩個物流運輸網範例。邊上的時間標註為發車時間；為便於說明，假設每段車程為一小時。在左圖中，貨物於 15:00 自 a 出發，16:00 抵達 b ；再於 17:00 自 b 出發，18:00 抵達 c 。在右圖中，由於 b 到 c 的發車時間為 10:00 (早於 16:00 抵達 b 的時刻)，故今日無法自 a 送達 c 。

(一) 定義

A. 時變圖、時變路徑與時變連通性

一個時變圖 (G, λ) 由一個底圖 (underlying graph) $G = (V, E)$ 與一個標籤函數 $\lambda: E \rightarrow 2^{\mathbb{N}}$ 組成 (在此計畫書中，我們假設 $\mathbb{N} = \{1, 2, 3, \dots\}$)，其中 V 為節點集合， E 為邊集合， $\lambda(e)$ 則表示邊 $e \in E$ 的可用時間 (亦稱標籤，label) 集合。

時變圖的核心概念為時變路徑 (temporal path)。如圖 1 所示，為使起點能抵達終點，路徑上各邊的時間標籤必須嚴格遞增。更明確地說， (G, λ) 中的一條時變路徑可表示為序列 $(e_1, t_1), (e_2, t_2), \dots, (e_\ell, t_\ell)$ ，並需滿足以下限制：

- **路徑連通性**： e_i 為 G 的邊，且 $e_1 e_2 \dots e_\ell$ 構成 G 中的一條路徑。
- **時間遞增性**： $t_i \in \lambda(e_i)$ ，且 $t_1 < t_2 < \dots < t_\ell$ 。

上述「時間遞增」限制隱含了「通過每條邊所需時間為 1」的假設。若令 $e_i = (v_i, v_{i+1})$ ，則從 v_i 出發的時間為 t_i ，並在時間 $t_i + 1$ 抵達 v_{i+1} ；因此下一段出發時間 t_{i+1} 必須大於等於 $t_i + 1$ 。若 G 中任兩節點間皆存在時變路徑，則稱 (G, λ) 為時變連通 (temporally connected)。

圖 2 給出一個時變連通例子與一個反例。需注意：即便存在由 u 到 v 的時變路徑，反向由 v 到 u 的時變路徑未必存在；例如圖 2 右圖中，存在由 d 至 c 的時變路徑 $((d, a), 1), ((a, b), 2), ((b, c), 5)$ ，但不存在由 c 至 d 的時變路徑。

B. Carnevale 等人定義之 MAL 問題

本計畫首先回顧 Carnevale 等人提出的 Minimum Aged Labelling (MAL) 問題 [CDO25]。於大眾或物流運輸網中，發車常有最晚截止時間限制 (例如晚間 23:00 後停駛)，此限制對應到時間標籤或可用時間的上限 (upper bound)。Carnevale 等人稱此上限為 maximum allowed age，並定義 MAL 問題：在滿足 maximum allowed age 情況下，以最少標籤數 (可視為最少班次) 確保時變連通。



圖 2: 兩個時變圖範例。這兩個時變圖的唯一不同標籤在於邊 (b, a) ，左圖邊 (b, a) 的標籤為 2，右圖對應的標籤為 3。左圖為時變連通，任何節點皆存在時變路徑抵達任意其他節點。舉例來說， c 到 d 存在一個時變路徑 $((c, b), 1), ((b, a), 2), ((a, d), 3)$ 。右圖不為時變連通。舉例來說，右圖不存在由 c 抵達 d 的時變路徑。

定義 1: Minimum Aged Labelling (MAL) 問題 [CDO25]

輸入： 一個圖 $G = (V, E)$ ，與最大標籤限制 a 。

輸出： 標籤函數 $\lambda: E \rightarrow 2^{\mathbb{N}}$ 。

限制：

1. **時變連通限制：** (G, λ) 必須為時變連通。
2. **最大標籤限制：** 對於每個邊 $e \in E$ ，最大標籤不超過 a 。換句話說，對於所有 $e \in E$ ， $\lambda(e) \subseteq \{1, \dots, a\}$ 。

目標： 最小化標籤總數：

$$\min \sum_{e \in E} |\lambda(e)|。$$

定義 $\text{Diam}(G)$ 為圖 G 的直徑 (diameter)，即為 G 中最遠的兩點間最短路徑長度。一個關於 MAL 問題的簡單觀察如下：為了讓 MAL 問題有可行解 (feasible solution)，最大標籤限制 a 至少需為 $\text{Diam}(G)$ (否則最遠的兩個點間不存在時變路徑)。

C. 本計畫擬探討之第一個延伸問題：具拉伸程度限制之 MAL 問題 (MAL-S 問題)

C.1 時變路徑長度與傳輸延遲之探討

在此研究計畫中，我們將從兩個維度延伸 MAL 問題：一是引入邊的權重 (edge weight) 以貼近實際運輸場景，二是引入路徑長度的考量。Carnevale 等人提出的 MAL 問題僅關注「連通性」與「標籤總數」，卻忽略了路徑的傳輸長度。在極端情況下，為了節省標籤成本，演算法所規劃的時變路徑可能迫使貨物在站點進行極長時間的等待，或是繞行很遠的距離，導致實際運送時間遠高於物理上的最短距離。舉例來說，當 a 足夠大時，Carnevale 等人提出的演算法會先建構一個生成樹 (spanning tree)，接著在生成樹的邊上指定標籤。這樣的作法可能會讓兩個葉節點有很長的時變路徑，但是在底圖上他們可能有很近的距離。為避免此情形，必須對時變路徑的長度加以限制。

在定義時變路徑長度之前，我們首先需推廣時變圖的底圖。現有文獻如 [KMMS24, KMMS25, BDG⁺22, MMMS25b] 多假設，「通過邊的時間為 1 (unit time)」，然而在真實物流或交通網中，不同路段的行駛時間 (travel time) 往往不同。因此，我們採用邊加權圖 (edge-weighted graph) $G = (V, E, w)$ 作為底圖，其中權重 $w(e)$ 代表通過邊 e 所需時間。此假設與時變圖的另一研究分支相同 [ABD⁺25, DPSW18, WCK⁺16, DOS12, JS24, WDdACG12]。

在此模型下，一條時變路徑 $p = ((e_1, t_1), (e_2, t_2), \dots, (e_k, t_k))$ 除了需滿足路徑連通性外，其時間標籤必須符合以下限制：

$$t_{i+1} \geq t_i + w(e_i)。$$

令 $e_i = (v_i, v_{i+1})$ ，此不等式隱含了抵達中繼點 v_{i+1} 的時刻為 $t_i + w(e_i)$ ，而下一段從 v_{i+1} 出發時刻 t_{i+1} 必須在此之後；兩者之差 $(t_{i+1}) - (t_i + w(e_i))$ 即為在 v_{i+1} 的等待時間。

基於上述模型，時變路徑 p 在時變圖 (G, λ) 的長度，以 $\text{dist}_{G, \lambda}(p)$ 表示，定義為「抵達終點時刻」減去「起點出發時刻」，亦即總傳輸時間：

$$\text{dist}_{G, \lambda}(p) = (t_k + w(e_k)) - t_1。$$

此定義不僅包含了所有邊上的行駛時間 $\sum_{1 \leq i \leq k} w(e_i)$ ，也包含了在各中繼站點的所有等待時間。這正是時變圖與靜態圖最大的差異所在：在時變圖中，路徑長度由邊的物理距離與標籤

的時間安排共同決定。

C.2 時變距離與拉伸程度

給定一個時變圖 (G, λ) ，任兩節點 s, t 在 (G, λ) 的**時變距離**，以 $\text{dist}_{G, \lambda}(s, t)$ 表示，定義為所有 (G, λ) 上從 s 到 t 的合法時變路徑中，長度最短者的長度：

$$\text{dist}_{G, \lambda}(s, t) = \min_{p: p \text{ 為 } (G, \lambda) \text{ 上從 } s \text{ 到 } t \text{ 的時變路徑}} \text{dist}_{G, \lambda}(p)。$$

另一方面，令 $\text{dist}_G(s, t)$ 為底圖 G 上僅考慮邊權重 w 的最短路徑距離（即傳統靜態最短路徑，不考慮等待時間）。由於時變路徑必然包含行駛時間，且可能包含等待時間，故恆有 $\text{dist}_{G, \lambda}(s, t) \geq \text{dist}_G(s, t)$ 。為了衡量時變標籤設計的優劣，我們引入拉伸程度（stretch）的概念，定義為時變距離相對於靜態距離的放大倍率。若我們要求任兩點間的傳輸效率不得過低，即相當於要求：

$$\frac{\text{dist}_{G, \lambda}(s, t)}{\text{dist}_G(s, t)} \leq \alpha，$$

其中 $\alpha \geq 1$ 為給定的拉伸容忍上限。此限制確保了貨物運送或旅客移動的總時間（含等待）不會超過理想時間的 α 倍。

C.3 MAL-S 問題定義

承襲 MAL 問題的精神，本問題的目標為最小化總發車數（即總標籤數）。我們稱此延伸問題為 Minimum Aged Labelling with Stretch constraints (MAL-S) 問題，其定義如下：

定義 2: Minimum Aged Labelling with Stretch constraints (MAL-S) 問題

輸入：

1. 一個圖 $G = (V, E)$ 與邊行駛時間函數 $w: E \rightarrow \mathbb{N}$ 。
2. 最大標籤限制 $a \in \mathbb{N}$ 。
3. 拉伸上限 $\alpha \geq 1$ 。

輸出： 標籤函數 $\lambda: E \rightarrow 2^{\mathbb{N}}$ 。

限制：

1. **時變連通限制：** (G, λ) 為時變連通。
2. **最大標籤限制：** 對於所有 $e \in E$ ， $\lambda(e) \subseteq \{1, \dots, a\}$ 。
3. **拉伸程度上限：** 對於任何節點對 $s, t \in V$ ，皆滿足 $\frac{\text{dist}_{G, \lambda}(s, t)}{\text{dist}_G(s, t)} \leq \alpha$ 。

目標： 最小化標籤總數：

$$\min \sum_{e \in E} |\lambda(e)|。$$

D. 本計畫擬探討之第二個延伸問題：具容量限制之 MAL 問題（MAL-C 問題）

在此計畫書前述的範例中，時變圖上的每一條時變邊（即每一個可用時間標籤）隱含對應到單一車次。然而，現實場景中節點間的載貨需求往往呈現異質性，因此單一邊在特定時刻

可能需要發出多輛車才能滿足需求。因此，問題的本質從單純的標籤指定，轉變為決定每個標籤對應的車輛數。

為了表示節點間需求的異質性，我們令需求集合為 $D = \{d_1, d_2, \dots, d_h\}$ 。對於每個需求 d_i ，我們定義四個屬性： $\text{src}(d_i)$ 、 $\text{dst}(d_i)$ 、 $\text{rel}(d_i)$ 與 $\text{dem}(d_i)$ ，分別代表該需求的起點、終點、產生時間（release time）與貨運需求量。

與 MAL-S 問題類似，我們考慮邊的行駛時間，令 $w(e)$ 表示通過邊 e 所需的時間。但在此問題中，為了簡化模型並聚焦於容量規劃，我們暫不考慮拉伸程度的限制，亦不另外訂定個別貨物的最晚抵達時間（deadline）。我們僅要求每個需求 d_i 必須在產生時間 $\text{rel}(d_i)$ 之後，透過我們設計的時變網路成功送抵終點。值得注意的是，由於最大時間標籤限制為 a ，所有需求抵達終點的時間自然受限於 $a + \max_{e \in E} w(e)$ 。

我們假設需求可以被分割（splittable flow），並以 $f_i(e, t)$ 代表需求 d_i 在時間 t 流入邊 e 的流量（該流量將於時刻 $t + w(e)$ 流入 e 的終點）。 f_i 在時變圖上需要滿足流量守恒，且總流量須達 $\text{dem}(d_i)$ 。特別需要說明的是，由於時變邊並非隨時可用，貨物可能需要在中繼點暫存，等待未來合適的發車時間。換言之，對於任意時間點 t 與中繼點 v ，流量需滿足：

$$(\text{時間 } t \text{ 流入 } v \text{ 的流量}) = (\text{時間 } t \text{ 流出 } v \text{ 的流量}) + (\text{時間 } t \text{ 於 } v \text{ 的暫存淨變動量})。$$

為了簡化討論，我們假設每個節點的暫存空間無限大。關於如何在數學上將上述時變圖上的流量與暫存限制轉換為一般圖的流量限制，我們將在第（五）B 小節詳細說明。

在此問題中，每個邊 e 在時間 t 的發車班次即為該邊總流量的無條件進位值： $\lceil \sum_{i=1}^h f_i(e, t) \rceil$ 。承襲 MAL 問題的精神，本問題的目標為最小化總發車數量。我們稱此延伸問題為 Minimum Aged Labelling with Capacity constraints (MAL-C)，其正式定義如下：

定義 3: Minimum Aged Labelling with Capacity constraints (MAL-C) 問題

輸入：

1. 一個圖 $G = (V, E)$ 與邊行駛時間函數 $w: E \rightarrow \mathbb{N}$ 。
2. 最大標籤限制 $a \in \mathbb{N}$ 。
3. 需求集合 $D = \{d_1, \dots, d_h\}$ ，其中 d_i 包含屬性 $\text{src}(d_i), \text{dst}(d_i), \text{rel}(d_i), \text{dem}(d_i)$ 。

輸出：

1. 標籤函數 $\lambda: E \rightarrow 2^{\mathbb{N}}$ 。
2. 流量函數族 $\{f_1, \dots, f_h\}$ ，其中 $f_i(e, t) \in \mathbb{R}_{\geq 0}$ 表示 d_i 在時間 t 流入邊 e 的流量（該流量將於時刻 $t + w(e)$ 流入 e 的終點）。

限制：

1. **標籤與流量一致性**：對於所有 $e \in E$ 、 $t \in \{1, \dots, a\}$ 與 $i \in \{1, \dots, h\}$ ，若 $f_i(e, t) > 0$ ，則必須滿足 $t \in \lambda(e)$ 。
2. **時變流量守恒**：對於每一個需求 d_i ，流量函數 f_i 須在時變圖上滿足流量守恒（包含節點暫存流動），且在終點累積的總流量需等於 $\text{dem}(d_i)$ 。
3. **最大標籤限制**：對於所有 $e \in E$ ， $\lambda(e) \subseteq \{1, \dots, a\}$ 。

目標：最小化總發車數量：

$$\min \sum_{e \in E} \sum_{t=1}^a \left\lceil \sum_{i=1}^h f_i(e, t) \right\rceil$$

E. 其他定義

在此計畫書中，我們定義 $n = |V|$ 為底圖中的節點數， $m = |E|$ 為底圖中的邊數。節點 v 的偏心率 (eccentricity)，以 $\text{Ecc}_G(v)$ 表示，為 v 到 G 中其他節點的最大最短路徑長度。更明確地說， $\text{Ecc}_G(v) = \max_{u \in V} \text{dist}_G(v, u)$ 。圖 G 的半徑 (radius)，以 $\text{Rad}(G)$ 表示，是 G 中最小的偏心率，也就是 $\text{Rad}(G) = \min_{v \in V} \text{Ecc}_G(v)$ 。

本計畫聚焦於近似演算法。以最小化問題為例，對任一輸入 I ，令最佳目標函數值為 $\text{OPT}(I)$ ，演算法 $\mathcal{A}$ 產生的目標函數值為 $\mathcal{A}(I)$ 。若對所有輸入 I ，演算法 $\mathcal{A}$ 皆滿足 $\mathcal{A}(I) \leq r \cdot \text{OPT}(I)$ ，則稱 $\mathcal{A}$ 為一個 r -近似演算法。若對任意常數 $\epsilon > 0$ ，存在一個在輸入大小的多項式時間內運行的演算法 $\mathcal{A}_\epsilon$ ，其輸出滿足 $\mathcal{A}_\epsilon(I) \leq (1 + \epsilon)\text{OPT}(I)$ ，則稱其為 PTAS (Polynomial-Time Approximation Scheme)。

(二) 相關文獻探討

在此小節中，我們會先介紹 MAL 問題的過去研究結果 [MMS19, KMMS24, CDO25]。接著探討時變圖在文獻上的其他相關研究。儘管 MAL 問題已有初步研究成果，但現有文獻多聚焦於單純的連通性與標籤數量最小化，尚未有研究將拉伸程度限制與邊容量限制結合至 MAL 問題的框架中。填補此一研究缺口，即為本計畫擬探討 MAL-S 與 MAL-C 問題之核心動機。

A. MAL 問題的既有研究結果

雖然本計畫主要受 Carnevale 等人在 MAL 問題上的成果啟發，實際上 MAL 問題最早由 Mertzios 等人探討 [MMS19]。他們首先證明：即使 $a = \text{Diam}(G)$ ，在有向底圖上 MAL 仍為 APX-hard；換言之，當 $a = \text{Diam}(G)$ 時，有向圖上不存在 PTAS。同時，Mertzios 等人於 $a = \text{Diam}(G)$ 的設定下提出了一個 $(n - 1)$ -近似演算法。另一方面，當底圖為 DAG (Directed Acyclic Graph) 時，Klobas 等人給出了多項式時間演算法 [KMMS24]；他們亦證明：即使底圖為無向圖且 $a = \text{Diam}(G)$ ，MAL 問題依然是 NP-hard。

近期，Carnevale 等人針對無向圖建立了一系列近似比的上下界 [CDO25]。令 OPT 表示 MAL 問題的最小標籤總數，Carnevale 等人針對不同 a 值範圍所提出的近似比上界與多項式時間演算法結果彙整於表 1。

最大標籤限制 (a 的條件)	近似比 / 結果
任意 $a \geq \text{Diam}(G)$	$O(\text{Diam}(G)n^{3/5+\epsilon})$
$a \geq \text{Diam}(G) + 2$	$O(\text{Diam}(G)n^{1/2})$
$a \geq \text{Diam}(G) + 4$	$O(\text{Diam}(G)n^{2/5})$
$a \geq \text{Diam}(G) + 6$	$O(\text{Diam}(G)n^{1/3})$
$a \geq \lceil 3/2 \text{Diam}(G) \rceil$	$O(\sqrt{n \log n})$
$a \geq \lceil 5/3 \text{Diam}(G) \rceil$	$O(\sqrt[3]{\text{Diam}(G)n \log^2 n})$
$a \geq 2 \text{Rad}(G)$	$\text{OPT} + 2$ (多項式時間)
$a \geq 2 \text{Rad}(G) + 1$	$\text{OPT} + 1$ (多項式時間)
$a \geq 2 \text{Diam}(G) + 2$	最佳解 (多項式時間)

表 1: Carnevale 等人針對無向圖的 MAL 問題近似比與複雜度結果彙整 [CDO25]。

在下界方面，於常見複雜度假設下，有以下無法近似 (inapproximability) 結果。假設 $P \neq NP$ ，即使 $a = \text{Diam}(G) = 2$ ，對所有 $0 < \epsilon < \frac{1}{4}$ ，MAL 問題不存在 $(\epsilon \log n)$ -近似演算法。若再進一步假設 $NP \not\subseteq \text{DTIME}(2^{\text{polylog}(n)})$ ，則即使 $a = \text{Diam}(G) \geq 3$ ，對任何 $0 < \epsilon < 1$ ，MAL 問題亦不存在 $2^{\log^{1-\epsilon} n}$ -近似演算法。

總結上述無向圖相關結果如下：

1. 當 $a = \text{Diam}(G)$ 時，MAL 問題為 NP-hard。
2. 當 a 介於 $\text{Diam}(G) + 1$ 與 $2 \text{Diam}(G) + 1$ 之間時，其複雜度仍不明。
3. 當 $a \geq 2 \text{Diam}(G) + 2$ 時，MAL 問題可於多項式時間內求解。

Carnevale 等人據此提出下列開放問題 (Open Problem)：

Open Question 1: [CDO25]

當 $\text{Diam}(G) + 1 \leq a \leq 2 \text{Diam}(G) + 1$ 且底圖為無向圖時，MAL 問題是否為 NP-hard？

B. 無最大標籤限制的 MAL 問題相關研究

Klobas 等人亦研究了無最大標籤限制的 MAL 問題變形 [KMMS24]，亦即可使用的標籤不設上界（或 $a = \infty$ ）。在此設定下，當底圖為無向圖或 DAG 時，最佳解可以在多項式時間求得 [KMMS24]。此外，Carnevale 等人針對 $a \geq 2 \text{Diam}(G) + 2$ 所設計的演算法，同樣適用於不設標籤上限的情況求得最佳解 [CDO25]。

整體而言，若拿掉最大標籤限制且底圖為無向圖，則 MAL 問題屬於 P；若拿掉最大標籤限制且底圖為有向圖，其複雜度仍屬未知，目前亦沒有已知的近似演算法。對此，Klobas 等人提出下列開放問題：

Open Question 2: [KMMS24]

若不考慮最大標籤限制，且底圖為有向圖時，MAL 問題是否存在多項式時間演算法，或是近似演算法？

C. 時變距離計算演算法

在 MAL-S 問題定義中，我們考慮了時變距離（最短時變路徑之長度）。對於離散標籤（每條邊的可用時間為離散集合）的情形，一系列研究 [WCK⁺16, DPSW18, BHNN20] 提出了時間複雜度為 $\tilde{O}(\sum_{e \in E} |\lambda(e)|)$ 的演算法（其中 $\tilde{O}(\cdot)$ 省略了 $\text{polylog}(n)$ 因子）。另一方面，若可用時間為連續區間的集合，Dehne 等人給出了時間複雜度為 $\tilde{O}(|E| \sum_{e \in E} |\lambda(e)|)$ 的演算法 [DOS12]。其後 Aubian 等人對該模型建立了時間複雜度下界：在 APSP 猜想ⁱ下，對於任意 $\epsilon > 0$ ，不存在時間複雜度 $O((n \sum_{e \in E} |\lambda(e)|)^{1-\epsilon})$ 的演算法 [ABD⁺25]。

D. 考慮拉伸程度的 λ 函數設計

在 MAL-S 問題中，我們為每個節點對加入了拉伸程度的上限；與此相近，Mertzios 等人研究了一個直接最小化拉伸程度的問題 [MMMS25b]。他們考慮週期為 Δ 的單標籤週期

ⁱAPSP 猜想：對任意常數 $\epsilon > 0$ ，All-Pairs Shortest Paths 問題不存在時間複雜度為 $O(n^{3-\epsilon})$ 的演算法。

性時變圖：每條邊被指派恰一個標籤 t ，並在時間 $t, t + \Delta, t + 2\Delta, \dots$ 可用。該問題要求輸出標籤指派函數 $\lambda: E \rightarrow \mathbb{N}$ 以最小化拉伸程度。在近似比下限方面，他們證明（假設 $P \neq NP$ ）：對任意 $0 < \epsilon < 1$ 與任意 $c > 1$ ，不存在 $\Delta^{1-\epsilon}$ -近似演算法，亦不存在 2^{n^c} -近似演算法 [MMMS25b]。另一方面，他們提出了一個多項式時間演算法，保證拉伸程度至多為 $\Delta - \frac{\Delta-1}{\min\{\text{Rad}(G)+1, \text{Diam}(G)\}}$ 。需特別說明的是，上述結果均在無向且無邊權重（無延遲）的底圖設定下建立。

Mertzios 等人同樣考慮週期為 Δ 的單標籤時變圖，但將底圖限制為樹（Tree），並額外給定一個距離上界矩陣 D 。目標是輸出標籤指派函數 $\lambda: E \rightarrow \mathbb{N}$ ，使得所有節點對 (v_i, v_j) 皆滿足 $\text{dist}_{G,\lambda}(v_i, v_j) \leq D_{i,j}$ [MMMS25a]。他們證明此問題為 NP-hard。Klobas 等人 [KMMS25] 研究了一個相近的問題：他們要求 $\text{dist}_{G,\lambda}(v_i, v_j) = D_{i,j}$ （時變距離矩陣恰為 D ）。相對於 Mertzios 等人考慮的問題，Klobas 等人考慮的問題限制十分嚴格，反而可以利用這些限制在多項式時間求解。

進一步地，Erlebach 等人 [EMW25] 放寬為每條邊至多可指定多個標籤的情形。他們證明了若每個邊可以使用 5 個標籤時，問題仍為 NP-hard。另一方面，若考慮非週期的時變圖，且 D 的所有數字皆不為無限大、每個邊可用標籤數至少為 n^2 ，並且底圖 G 具有所謂的 p -路徑多樣性（即對任意兩點 (u, v) ， u 至 v 路徑數至多為 p ），則此問題可在 $O(pn^4)$ 時間內求解。由於樹圖與環圖皆滿足 $p = O(1)$ （樹為 $p = 1$ ，無向單環為 $p = 2$ ），故在此兩類底圖上可得到多項式時間演算法。

E. 時變圖 Spanner 相關研究

與拉伸程度緊密相關的是 Spanner 問題。在傳統的 Spanner 問題中，輸入為靜態圖 G 與拉伸程度上限 $\alpha \geq 1$ ，輸出為 G 的子圖（subgraph） H ，並要求任兩點在 H 的距離至多為其在 G 的距離的 α 倍；目標是最小化 H 的邊數。在時變圖情境中，Bilò 等人 [BDG⁺22] 將上述設定推廣為時變圖 Spanner。輸入為時變圖 (G, λ) 與拉伸上限 α ；輸出為時變圖 (H, λ') ，其中 H 是 G 的子圖，且對每條 H 的邊 e ， $\lambda'(e) = \lambda(e)$ （不重新指派時間標籤）。限制為對所有節點 u, v ，

$$\text{dist}_{H,\lambda'}(u, v) \leq \alpha \text{dist}_{G,\lambda}(u, v)。$$

目標同樣是最小化 H 的邊數。

與 MAL 問題定義不同，Bilò 等人 [BDG⁺22] 對時變路徑的標籤只要求非遞減（non-decreasing），因此一條時變路徑上的所有邊可以使用相同的標籤；此外，他們的輸入時變圖假設每條邊僅有一個標籤。當底圖為完全圖時，對任意正整數 k ，他們構造出一個拉伸 $\alpha = 2k - 1$ 、邊數為 $O(kn^{1+1/k} \log^{1-1/k} n)$ 的時變圖 Spanner；值得注意的是，取 $k = \lfloor \log n \rfloor$ ，可得邊數為 $O(n \log^2 n)$ 的時變圖 Spanner。若僅要求時變連通（即 $\alpha = \Theta(n)$ ），且底圖 G 為完全圖（Complete Graph）時，Casteigts 等人給出邊數為 $O(n \log n)$ 的時變圖 Spanner [CPS21]；進一步地，Angrick 等人於特定的標籤配置下（底圖仍為完全圖）建構了邊數為 $O(n)$ 的時變圖 Spanner [ABF⁺24]。

然而，當底圖不是完全圖時，要構造邊數很少的時變圖 Spanner 變得困難得多。更具體地說，Kempe 等人證明了：存在某些底圖不為完全圖的時變圖 (G, λ) ，即便只要求輸出一個時變連通的 Spanner（等價於 $\alpha = \Theta(n)$ ），所需邊數仍達 $\Theta(n^2)$ [KKK02]。

此外，經典的時變 Spanner 問題是：在給定時變連通的時變圖 (G, λ) 中，選出最少的邊形成子圖 H ，以滿足時變距離限制。與此相對，Bellitto 等人 [BPE25] 研究的是一個擴增（Augmentation）版本的問題：輸入的 (G, λ) 不一定時變連通，並且另給一組候選邊集合 F ，每條候選邊的可行時間標籤已事先固定；目標是在 F 中選出最小子集 $F' \subseteq F$ ，使得 (G, λ) 加上 F' 成為時變連通圖。他們證明此問題為 NP-hard。

F. 考慮容量限制的時變圖相關研究

與本計畫的 MAL-C 問題最接近的是 Akrida 等人的研究 [ACG⁺19]：他們在時變圖上引入邊的容量；給定起點 s 、終點 t 與截止時間 T ，目標是最大化在時間 T 前可由 s 傳到 t 的總流量，並提出了多項式時間演算法。需要注意的是，該模型假設每條邊的延遲皆為 1，因此未處理任意的邊延遲。相較之下，經典的最快轉運 (Quickest Transshipment) 問題在靜態圖上同時考慮邊容量與任意整數延遲，但是此問題的邊在任意時間皆可用。此問題可於多項式時間內求解 [FF58]，而 Hoppe 與 Tardos 更將之擴展至多起點終點的情境，同樣給出多項式時間演算法 [HT00]。

在應用面，Tao 等人 [TMGV23] 以衛星-地面站-雲端組成的時變圖為例，建立具頻寬上限的模型：衛星僅在特定時段能與地面站通訊，地面站到雲端的上傳頻寬也受限；此外，每顆衛星同時最多僅能連到一個地面站，使得問題兼具匹配 (matching) 與流量 (flow) 決策，研究以實驗評估吞吐量最大化演算法的效能。

上述研究多假設可用時間標籤與容量已給定，並以最大化可運送流量為目標；Akrida 等人亦未處理一般化邊延遲。相對地，MAL-C 問題要求同時設計各邊在各時刻的可用性 (標籤) 與對應容量，以滿足給定的多對需求並最小化總容量 (總發車數)；此外，我們亦允許導入相異的邊延遲以貼近運輸實務。

(三) Carnevale 等人提出的無向底圖演算法框架

在 [CDO25] 中，Carnevale 等人提出的證明隱含了若干關於 MAL 的演算法。在此小節中，我們會將這些演算法推廣成演算法框架 (我們稱其為 Carnevale 演算法框架)。我們目前一些初步的研究成果與未來的研究規劃即是基於這些演算法框架。我們主要討論四個演算法框架：

1. 第一個 Carnevale 演算法框架首先計算一個生成樹 (spanning tree) T ，令其根節點為 r 。接著我們在 T 上給定時間標籤，讓每個節點可以先抵達 r ，接著再從 r 抵達任何節點。此演算法框架適用於 a 足夠大的情形。
2. 第二個 Carnevale 演算法框架首先將 MAL 問題轉成一個與 Spanner 相關的問題，再利用 Spanner 演算法求得 MAL 的解。此演算法框架適用於 $\text{Diam}(G)$ 足夠小的情形。
3. 第三與第四個 Carnevale 演算法框架首先計算一個 Dominating Set S ，並確保所有節點到 S 的距離都不會太長。接著讓每個節點透過時變路徑抵達 S ，最後再由 S 透過時變路徑抵達任意節點。

上述四個框架主要都是基於無向底圖設計。除了第一個框架外，後三者在 [CDO25] 中都有提到如何延伸至有向底圖。我們在第 (四) 小節中，會討論如何將第一個框架延伸至有向底圖。

A. 第一個 Carnevale 演算法框架：基於生成樹的標籤指定演算法

在這個演算法中，我們首先計算底圖的生成樹 T ，接著找到在 T 上距離根節點 r 最遠的節點，稱其為 z 。我們用 $\text{dist}_T(u, v)$ 代表 u 到 v 在 T 上的路徑長度 (邊個數)。因此 $z = \arg \max_{v \in V} \text{dist}_T(r, v)$ 。一個簡單的觀察如下：如果要讓 z 透過時變路徑抵達 r ，我們可以從 z 出發，沿著 T 走向 r ，經過的邊依序指定 $1, 2, 3, \dots, \text{dist}_T(z, r)$ 。反過來說，我們可以由 r 出發，沿著 T 走向 z ，經過的邊依序指定 $\text{dist}_T(z, r), \text{dist}_T(z, r) - 1, \text{dist}_T(z, r) - 2, \dots, 1$ 。不難觀察得到，我們可以延伸此想法，沿著 T 由 r 出發往外走，由內而外將邊指定標籤

Algorithm 2 第二個 Carnevale 演算法框架：基於 DCSS 問題的標籤指定演算法

Input: $G = (V, E)$ 、最大標籤限制 a 、DCSS 問題演算法 $\mathcal{A}$

- 1: 建立 DCSS 問題實例 $\mathcal{I}$ ，其中輸入圖為 G ，最大直徑限制為 a
 - 2: $H \leftarrow \mathcal{A}(\mathcal{I})$
 - 3: 對於每個 $e \in E(H)$ ， $\lambda(e) = \{1, 2, \dots, a\}$
-

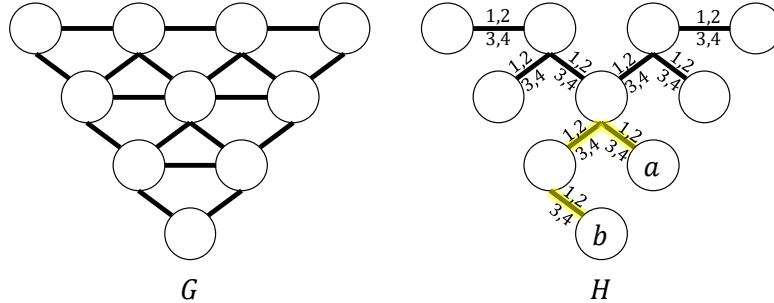


圖 4: Algorithm 2 範例。

c 點後，需要等到時間 13 才能繼續前往目的地。事實上，即使在樹上的距離很短，等待時間也可能很長（如 x 到 y 的時變路徑）。在此研究計畫中，我們希望透過研究 MAL-S 問題解決上述路徑過長與等待時間過長的缺陷。

B. 第二個 Carnevale 演算法框架：將 MAL 問題轉換成 DCSS 問題

為了介紹此方法，我們首先定義 Diameter Constrained Spanning Subgraph (DCSS) 問題。對於任意圖 H ，我們用 $E(H)$ 代表 H 的邊集合。

定義 4: Diameter Constrained Spanning Subgraph (DCSS) 問題

輸入： 一個圖 $G = (V, E)$ ，與最大直徑限制 b ($b \geq \text{Diam}(G)$)。

輸出： G 的一個生成子圖 (spanning subgraph) H 。

限制： H 的直徑不超過 b ，即 $\text{Diam}(H) \leq b$ 。

目標： 最小化 H 的邊數：

$$\min |E(H)|$$

第二個演算法框架如下：對於任意的 MAL 問題輸入 G 與 a ，我們建構一個 DCSS 問題實例 $\mathcal{I}$ ，其輸入圖為 G ，最大允許直徑為 a 。令 $\mathcal{A}$ 為任何一個 DCSS 問題的演算法。我們利用 $\mathcal{A}$ 解決上述 DCSS 問題實例 $\mathcal{I}$ ，並令 $H = \mathcal{A}(\mathcal{I})$ 為 $\mathcal{A}$ 的輸出。我們接著建構一個 MAL 問題的解 λ 。 λ 的建構方法如下：對於子圖 H 中的每一條邊 $e \in E(H)$ ，我們賦予其完整的標籤集合 $\{1, 2, \dots, a\}$ 。亦即：

$$\lambda(e) = \{1, 2, \dots, a\}, \forall e \in E(H)。$$

上述演算法的虛擬程式碼呈現於 Algorithm 2。

此建構能確保時變連通性：因為 H 的直徑不超過 a ，故任兩節點 u, v 之間必存在一條長度 $k \leq a$ 的路徑 $P = e_1, e_2, \dots, e_k$ 。我們可以使用時變路徑 $(e_1, 1), (e_2, 2), \dots, (e_k, k)$ 來達成時變連通。在此轉換下，標籤總數為 $a|E(H)|$ 。

B.1 Algorithm 2 近似比分析

為方便討論，對於任何標籤函數 λ ，定義 $|\lambda| = \sum_{e \in E} |\lambda(e)|$ 。假設 $\mathcal{A}$ 為 DCSS 問題的 γ -近似演算法。接下來我們說明 Algorithm 2 對於 MAL 問題的近似比為 $a\gamma$ 。令 λ^* 為 MAL 問題在輸入 G, a 的最佳解、 H^* 為 DCSS 問題在輸入 G, a 的最佳解。我們稍後說明 $|E(H^*)| \leq |\lambda^*|$ 。基於此不等式，我們可以推導 Algorithm 2 的近似比如下：

$$|\lambda| = a|E(H)| \leq \gamma a|E(H^*)| \leq \gamma a|\lambda^*|,$$

其中第一個等式來自 Algorithm 2 的性質，第一個不等式來自於 $\mathcal{A}$ 為 DCSS 問題的 γ -近似演算法。根據上面推導，Algorithm 2 的近似比為 $a\gamma$ 。

接著我們解釋 $|E(H^*)| \leq |\lambda^*|$ 。令 H' 為邊集合 $\{e | \lambda^*(e) \neq \emptyset\}$ 建構的圖。不難觀察到 H' 是一個 DCSS 問題在輸入 G, a 的可行解 (feasible solution)，因此 $|E(H^*)| \leq |E(H')|$ 。另一方面， $|E(H')| \leq |\lambda^*|$ 顯然成立。結合上述兩個不等式，我們便有 $|E(H^*)| \leq |E(H')| \leq |\lambda^*|$ 。

B.2 DCSS 問題近似演算法

關於 DCSS 問題，Carnevale 等人提出了以下近似演算法：

1. Dodis 與 Khanna 研究的 Mincost Diameter- b 問題為 DCSS 問題的推廣。因此，Dodis 與 Khanna 針對 $b = 2$ 提出的 $O(\log n)$ -近似演算法可作為當 $b = 2$ 時 DCSS 問題的 $O(\log n)$ -近似演算法 [DK99]。
2. Chlamtác 等人研究的 Pairwise Spanner 問題也是 DCSS 問題的推廣。因此，Chlamtác 等人針對 Pairwise Spanner 問題提出的 $O(n^{3/5+\epsilon})$ -近似演算法可作為 DCSS 問題的 $O(n^{3/5+\epsilon})$ -近似演算法 [CDKL20]。
3. Additive- c Spanner 問題要找到邊數最小的生成子圖 H ，並確保任兩點在 H 距離至多為原圖距離加 c 。因此，Additive- c Spanner 演算法可以作為 $b = \text{Diam}(G) + c$ 的 DCSS 演算法。
 - Aingworth 等人設計的演算法可以用 $O(n^{3/2})$ 個邊建構 Additive-2 Spanner [ACIM99]。因為 DCSS 問題最佳解至少需使用 $n - 1$ 個邊，因此 Aingworth 等人設計的演算法可視為 DCSS 問題在 $b = \text{Diam}(G) + 2$ 時的 $O(n^{1/2})$ -近似演算法。
 - Bodwin 設計的演算法可以用 $O(n^{7/5})$ 個邊建構 Additive-4 Spanner [Bod22]。因此 Bodwin 設計的演算法可視為 DCSS 問題在 $b = \text{Diam}(G) + 4$ 時的 $O(n^{2/5})$ -近似演算法。
 - Baswana 等人設計的演算法可以用 $O(n^{4/3})$ 個邊建構 Additive-6 Spanner [BKMP10]。因此 Baswana 等人設計的演算法可視為 DCSS 問題在 $b = \text{Diam}(G) + 6$ 時的 $O(n^{1/3})$ -近似演算法。

B.3 Algorithm 2 缺陷

圖 4 顯示一個 Algorithm 2 範例。可以看出每個 H 的邊都有 a 個標籤。這也顯示了這個演算法的缺點：標籤數可能過多。另一方面，與 Algorithm 1 相同，對於某些在原圖距離很短的節點（例如 a 與 b ），他們有很長的時變路徑。所以 Algorithm 2 也無法有效的解決 MAL-S 問題。

C. 第三個 Carnevale 演算法框架：基於 Dominating Set 的標籤指定演算法

Algorithm 1 的直觀想法是設定單一中繼站，稱之為 r ，並強制任兩節點 u, v 之間的時變路徑都必須經過 r ，即 $u \rightsquigarrow r \rightsquigarrow v$ 的形式。以此簡化標籤結構，盡可能地共用標籤。

然而，這個策略存在根本性的缺陷。若以 r 為根建立一個 Shortest Path Tree，則距離 r 最遠的葉節點 u ，其與 r 之距離 $\text{dist}_T(u, r)$ ，在最壞情況下可能逼近圖的直徑 $\text{Diam}(G)$ 。這導致 $u \rightsquigarrow r \rightsquigarrow v$ 的路徑總長度 $\text{dist}_T(u, r) + \text{dist}_T(r, v)$ 在最壞情況下可能高達 $2 \text{Diam}(G)$ 。由於時變路徑標籤必須嚴格遞增，最大標籤值 a 必須至少為這條最長路徑的長度。因此，Algorithm 1 的最大缺陷是只能適用於 a 夠大的情形。

C.1 改善方向：以 Dominating Set 作為中繼站集合

單一中繼站方法之所以失敗，是因為它無法有效控制節點到中繼站的距離，因此我們試著拓展中繼站的大小。核心概念演變為：與其找一個節點 r 讓所有節點都離 r 近，不如找一個節點集合 S ，使得每一個節點 v 都能保證離 S 中的某個節點非常近。此概念即為 d -Dominating Set。

更明確地說，對於任何集合 $S \subseteq V$ 與任何節點 $v \in V$ ，我們定義 $\text{dist}_G(v, S)$ 為 v 在底圖 G 上到 S 任何一點的最短距離。換句話說 $\text{dist}_G(v, S) = \min_{s \in S} \text{dist}_G(v, s)$ 。對於任何底圖 $G = (V, E)$ 與任何集合 $S \subseteq V$ ，如果任何 G 中節點 v 到 S 的距離 ($\text{dist}_G(v, S)$) 至多為 d ，則我們稱 S 為 G 的 d -Dominating Set。舉例來說，傳統的 Dominating Set 即為 1-Dominating Set。

在我們正式介紹演算法前，我們先定義 Shortest Path Forest：給定一個根節點集合 $S \subseteq V$ ，一個底圖 G 上以 S 作為根節點集合的 Shortest Path Forest F ，是一個 G 的 Spanning Forest，並滿足對任意點 $v \in V$ ， $\text{dist}_G(v, S) = \text{dist}_F(v, S)$ 。因此，透過 F ，我們可以讓每個節點依據最短路徑，抵達 S 集合中距離自己最近的節點。

C.2 標籤指定演算法

對於底圖任意二節點 u, v ，我們的目標是讓 u 先以 Shortest Path Forest F 進入 d -Dominating Set S 。令 $r \in S$ 為 u 透過 F 抵達 S 的節點。我們接著以 r 為根建立 Shortest Path Tree T_r ，最後透過 T_r 抵達終點 v 。標籤指定方面，Shortest Path Forest F 上的每個樹 T 的標籤指定方式與 Algorithm 1 的第 4 行相同（讓 T 上節點抵達 T 的根節點）， T_r 上的標籤指定方法與 Algorithm 1 的第 5 行相同（讓 r 抵達 G 的每個節點）。演算法虛擬程式碼呈現於 Algorithm 3。

C.3 Algorithm 3 圖例

在圖 5 中， $S = \{a, b, c, d\}$ 為一個 3-Dominating Set，較粗的邊為以 S 為根節點集合的 Shortest Path Forest。我們用無底線數字表示第一階段的指定標籤。可以看出這些標籤由 S 往外遞減，確保每個節點皆可以透過時變路徑抵達 S 。我們用彩色底線數字表示第二階段的指定標籤，其中紅色、藍色、紫色、綠色分別代表 T_a 、 T_b 、 T_c 、 T_d 在第二階段指定的標籤。我們以 a 為例，解釋 a 如何透過 Shortest Path Tree T_a 抵達 V 中所有節點。可以看出標籤由 a 往外遞增，確保 a 可以透過時變路徑抵達 V 中所有節點。另一方面，因為第一階段抵達 a 的標籤為 2 ($\ell_a = 2$)，因此第二階段的標籤由 $\ell_a + 1 = 3$ 開始指定，確保 F_a 中的節點在第一階段透過 F_a 抵達 a 後，可以在第二階段繼續透過時變路徑抵達 V 中所有節點。

Theorem 1: [CDO25,CG20]

對於任何 $0 < \delta < 1$ 與 $n_2 < n$ ，以下敘述中，至少一個成立：

1. G 存在一個大小為 $O(\frac{n \log n}{n_2})$ 的 $\lfloor \delta \text{Diam}(G) \rfloor$ -Dominating Set S_1 ，且 S_1 可以在多項式時間求得。
2. G 存在一個大小為 n_2 的 $\lfloor (1 - \delta) \text{Diam}(G) \rfloor$ -Dominating Set S_2 ，且 S_2 可以在多項式時間求得。

令 $\delta = \frac{1}{2}$ ， $n_2 = \sqrt{n \log n}$ ，則 Theorem 1 可得一個 $\lfloor 0.5 \text{Diam}(G) \rfloor$ -Dominating Set S ，且 $|S| = O(\sqrt{n \log n})$ 。因此我們有以下結果。

Theorem 2: [CDO25]

當 $a \geq \text{Diam}(G) + \lfloor 0.5 \text{Diam}(G) \rfloor$ 時，MAL 問題存在一個 $O(\sqrt{n \log n})$ -近似演算法。

D. 第四個 Carnevale 演算法框架：另一基於 Dominating Set 的標籤指定演算法

在 Algorithm 3 中，起點 u 根據 Shortest Path Forest F 抵達 S 中的某個節點 r 後，會直接透過 r 的 Shortest Path Tree 抵達終點 v 。Carnevale 等人提出的另一個方法如下：令 F_q 為 F 中包含終點 v 的樹，其根節點為 $q \in S$ 。在這個新的方法中，起點 u 同樣根據 Shortest Path Forest F 抵達 S 中的某個節點 r （第一階段），接著透過最短路徑抵達 q （第二階段），最後透過 F_q 抵達終點 v （第三階段）。演算法的虛擬程式碼請見 Algorithm 4。與 Algorithm 3 的差異之一在於指定 F_q 的標籤時（第三階段），我們需要考慮第二階段中，抵達 q 所用的最大標籤（以 ℓ'_q 表示）。

D.1 Algorithm 4 圖例

我們考慮與前述 Algorithm 3 相同的底圖與 3-Dominating Set。Algorithm 4 與 Algorithm 3 有相同的第一階段。圖 6 中彩色底線標籤為 Algorithm 4 的第二階段標籤指定結果。我們以 a 點為例解釋 Algorithm 4 的第二階段。我們從 a 到其他 S 中節點（ b, c, d ）的最短路徑上指定紅色底線標籤，使得 a 可以抵達 S 的其他節點。從 b, c, d 節點出發的最短路徑標籤分別用藍色、紫色、綠色標示。Algorithm 4 的最終執行結果顯示於圖 6。以 a 為例， $\ell'_a = 6$ 。因此在第三階段中， F_a 上的標籤至少為 7。

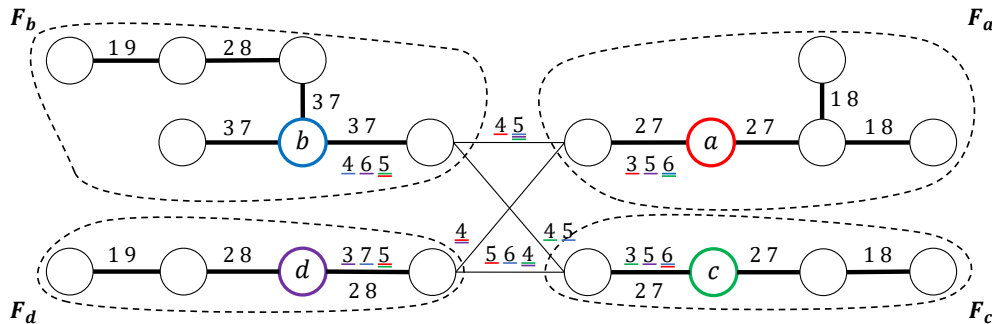


圖 6: Algorithm 4 範例。

Algorithm 4 第四個 Carnevale 演算法框架：另一基於 d -Dominating Set 的標籤指定演算法

Input: $G = (V, E)$ 與 d -Dominating Set S

第一階段：指定標籤讓所有節點利用 Shortest Path Forest 抵達 S

```
1: 計算  $G$  上以  $S$  為根節點集合的 Shortest Path Forest  $F$ 
2: for  $r \in S$  do
3:   令  $F_r$  為  $F$  中，以  $r$  為根的樹
4:    $z \leftarrow \arg \max_{v \in F_r} \text{dist}_{F_r}(r, v)$ 
5:    $\ell_r \leftarrow \text{dist}_{F_r}(r, z)$ 
6:   從  $r$  開始，沿著  $F_r$  往外指定標籤  $\ell_r, \ell_r - 1, \ell_r - 2, \dots, 1$ 
7: end for
第二階段：指定標籤讓  $S$  任兩節點可以透過最短路徑抵達對方
8: for  $r \in S$  do
9:   for  $q \in S \setminus \{r\}$  do
10:    令  $P = (v_1, v_2), (v_2, v_3), \dots, (v_{k-1}, v_k)$  為  $G$  中  $r$  到  $q$  的最短路徑，其中  $v_1 = r$  且  $v_k = q$ 
11:    從  $r$  開始，沿著  $P$  往  $q$  指定標籤  $\ell_r + 1, \ell_r + 2, \ell_r + 3, \dots, \ell_r + k - 1$ 
12:   end for
13: end for
第三階段：指定標籤讓  $S$  利用 Shortest Path Forest 抵達  $G$  中任意節點
14: for  $q \in S$  do
15:    $\ell'_q \leftarrow q$  相鄰邊上的最大標籤
16: end for
17: for  $q \in S$  do
18:   從  $q$  開始，沿著  $F_q$  往外指定標籤  $\ell'_q + 1, \ell'_q + 2, \ell'_q + 3, \dots$ 
19: end for
```

D.2 Algorithm 4 分析

不難看出 Algorithm 4 指定的標籤可以達成時變連通。第一階段與第三階段分別會讓最大標籤增加 d ，而第二階段會最多讓最大標籤增加 $\text{Diam}(G)$ 。因此這個方法適用於 $a \geq \text{Diam}(G) + 2d$ 的情形。

接著我們分析標籤數。因為 Shortest Path Forest F 有 $n - |S|$ 個邊，所以第一階段與第三階段分別指定了 $(n - |S|)$ 個標籤。因為任兩點最短路徑至多為 $\text{Diam}(G)$ 個邊，因此第二階段指定的標籤數最多為 $\frac{|S|(|S|-1)}{2} \text{Diam}(G)$ 。整體而言，標籤總數至多為 $2(n - |S|) + \frac{|S|(|S|-1)}{2} \text{Diam}(G)$ 。因此，Algorithm 4 的近似比為 $O(\frac{|S|^2 \text{Diam}(G)}{n})$ 。

D.3 結合 Algorithm 3 與 Algorithm 4

最後，我們說明 Carnevale 等人如何利用 Algorithm 3 與 Algorithm 4 在 $a \geq \text{Diam}(G) + 2\lfloor \frac{\text{Diam}(G)}{3} \rfloor$ 時，得到近似比為 $O(\sqrt[3]{\text{Diam}(G)n \log^2 n})$ 的 MAL 演算法。考慮 Theorem 1，若得到的是 S_1 ，則將 S_1 輸入 Algorithm 3；若得到的是 S_2 ，則將 S_2 輸入 Algorithm 4。ⁱⁱ

在設定 Theorem 1 的 δ 與 n_2 時，我們希望無論得到的是 S_1 或是 S_2 ，皆有同樣的最大標籤限制與近似比。我們首先考慮最大標籤限制，求解 $\text{Diam}(G) + \delta \text{Diam}(G) = \text{Diam}(G) + 2(1 - \delta) \text{Diam}(G)$ ，其中等式左邊為 Algorithm 3 結合 S_1 的最大標籤限制，右邊為 Algorithm 4

ⁱⁱ事實上，Carnevale 等人的方法是將 S_1 輸入 Algorithm 4， S_2 輸入 Algorithm 3。考慮到可讀性，我們選擇將 S_1 輸入 Algorithm 3， S_2 輸入 Algorithm 4。兩個做法有一樣的結果。

結合 S_2 的最大標籤限制。因此可得 $\delta = 2/3$ 與最大標籤限制 $\lceil \frac{5}{3} \text{Diam}(G) \rceil$ 。接著我們考慮近似比，求解 $\frac{n \log n}{n_2} = \frac{n_2^2 \text{Diam}(G)}{n}$ ，其中等式左邊為 Algorithm 3 結合 S_1 的近似比（亦為 S_1 的大小），右邊為 Algorithm 4 結合 S_2 的近似比。因此可得 $n_2 = \sqrt[3]{\frac{n^2 \log n}{\text{Diam}(G)}}$ 與近似比 $O(\sqrt[3]{n \log^2 n \text{Diam}(G)})$ 。我們因此有以下結果。

Theorem 3: [CDO25]

當 $a \geq \lceil \frac{5}{3} \text{Diam}(G) \rceil$ 時，MAL 問題存在一個 $O(\sqrt[3]{n \log^2 n \text{Diam}(G)})$ -近似演算法。

（四） 第一年研究方法：改良 MAL 演算法

此計畫第一年預計改善 MAL 問題的演算法，我們目前針對 $a < 1.5 \text{Diam}(G)$ 的情形已經有一些簡單的初步結果。在此節中，我們首先討論目前的初步結果，接著我們討論未來的研究方向。

A. 目前初步結果： $a < 1.5 \text{Diam}(G)$ 的近似演算法

回顧在第（三）C 小節中，我們討論了 Carnevale 等人的演算法框架 Algorithm 3。基於 Algorithm 3，我們首先設計一個當 $a < 1.5 \text{Diam}(G)$ 時的 $O(\frac{n}{\text{Diam}(G)})$ -近似演算法。當 $\text{Diam}(G)$ 愈大時，我們的演算法有愈好的近似比。另一方面，Carnevale 等人在 $a < 1.5 \text{Diam}(G)$ 的情形時，提出演算法的近似比為 $O(\text{Diam}(G)n^{3/5+\epsilon})$ （基於 Algorithm 2），適用於 $\text{Diam}(G)$ 較小的情形。結合上述兩個演算法，我們可以很容易得到一個近似比為 $O(n^{4/5+\epsilon})$ 的 MAL 演算法，注意到此近似比與 $\text{Diam}(G)$ 無關。

為了將 Algorithm 3 套用至 $a < 1.5 \text{Diam}(G)$ 的情形，我們需要一個 d -Dominating Set S ，其中 $d < 0.5 \text{Diam}(G)$ 。我們使用以下結果。

Theorem 4: [MM75]

對於任何包含 n 個節點的連通圖 G 與任何整數 $d < n$ ， G 存在一個大小為 $\lfloor \frac{n}{d+1} \rfloor$ 的 d -Dominating Set。

A.1 新 MAL 演算法

根據上述討論，對於任何 $d < n$ ，我們可以得到大小為 $O(\frac{n}{d})$ 的 d -Dominating Set。套用 Carnevale 等人的演算法框架 Algorithm 3，我們可以得到一個在 $a = \text{Diam}(G) + d$ 時，近似比為 $O(\frac{n}{d})$ 的 MAL 解 Sol_1 。因此我們有以下結果：

Theorem 5: 第一個初步結果

對於任何正整數 $d < n$ ，當 $a = \text{Diam}(G) + d$ 時，MAL 問題存在 $O(\frac{n}{d})$ -近似演算法。

接下來我們針對 $a < 1.5 \text{Diam}(G)$ 的情形討論。令 $a = (1+c) \text{Diam}(G)$ ，其中 $0 < c < 0.5$ 。我們設定 $d = c \text{Diam}(G)$ 。Theorem 5 可以得到一個近似比為 $O(\frac{n}{c \text{Diam}(G)})$ 的 MAL 解 Sol_1 。令 Sol_2 為 Carnevale 等人提出的 $O(\text{Diam}(G)n^{3/5+\epsilon})$ -近似演算法所得 MAL 解。我們有以下簡單觀察：

1. 當 $\text{Diam}(G) = \frac{1}{\sqrt{c}}n^{1/5-0.5\epsilon}$ 時， Sol_1 和 Sol_2 有相同的近似比 $O(\frac{1}{\sqrt{c}}n^{4/5+0.5\epsilon})$ 。
2. 當 $\text{Diam}(G) > \frac{1}{\sqrt{c}}n^{1/5-0.5\epsilon}$ 時， Sol_1 有較好的近似比。
3. 當 $\text{Diam}(G) < \frac{1}{\sqrt{c}}n^{1/5-0.5\epsilon}$ 時， Sol_2 有較好的近似比。

因此我們考慮以下演算法：當 $\text{Diam}(G) > \frac{1}{\sqrt{c}}n^{1/5-0.5\epsilon}$ 時，我們選擇 Sol_1 作為最終的 MAL 解；當 $\text{Diam}(G) \leq \frac{1}{\sqrt{c}}n^{1/5-0.5\epsilon}$ 時，我們選擇 Sol_2 作為最終的 MAL 解。這樣的演算法的近似比為 $O(\frac{1}{\sqrt{c}}n^{4/5+0.5\epsilon})$ 。因此，我們有以下結果：

Theorem 6: 第二個初步結果

對於任何 $0 < c < 0.5$ 與 $\epsilon > 0$ ，當 $a = (1 + c) \text{Diam}(G)$ 時，MAL 問題存在 $O(\frac{1}{\sqrt{c}}n^{4/5+0.5\epsilon})$ -近似演算法。

B. 目前初步結果：有向圖的 MAL 問題

回憶在 Open Question 2 中，Klobas 等人考慮有向的 MAL 問題，但是不考慮最大標籤限制。這個問題目前沒有已知的（近似）演算法。出乎意料的是，以下基礎結果尚未被文獻記載。

Theorem 7: 第三個初步結果

當底圖為強連通的有向圖，且 $a \geq 2 \text{Diam}(G)$ 時，MAL 問題有 2-近似演算法。

上述的演算法主要是延伸 Algorithm 1。我們首先任選一個節點 r ，並以 r 為根節點建立 in-branching T_{in} 與 out-branching T_{out} ：in-branching 指的是一個底圖的有向生成樹，且所有邊的方向都指向 r ；out-branching 指的是一個底圖的有向生成樹，且所有邊的方向都遠離 r 。我們利用 T_{in} 讓所有節點抵達 r ，接著再利用 T_{out} 讓 r 抵達所有節點。 T_{in} 的標籤指定方式如 Algorithm 1 的第 4 行， T_{out} 的標籤指定方式如 Algorithm 1 的第 5 行。這個方法可以保證時變連通。我們可以選擇 T_{in} 與 T_{out} 為 Shortest Path Tree，如此一來，最大標籤為 $2 \text{Diam}(G)$ 。另一方面，無論 T_{in} 與 T_{out} 的選擇為何，必定各自包含 $n - 1$ 邊，且每個邊使用一個標籤。因此標籤總數最多為 $2(n - 1)$ 。因為最佳解必定使用至少 $n - 1$ 個標籤，因此上述方法為 2-近似演算法。

C. 接續研究方向

前述的初步結果主要考慮 Algorithm 1 與 Algorithm 3 的延伸應用。我們預計針對另外兩個演算法框架 Algorithm 2 與 Algorithm 4 進一步改良。Algorithm 2 的明顯缺點是每個邊需要使用的標籤數過多，每個邊的標籤為 $1, 2, 3, \dots, a$ (Algorithm 2 第三行)。Algorithm 4 的明顯缺點是需要為 d -Dominating Set S 任兩點個別建立最短路徑，喪失標籤共用機會，進而導致標籤數上升。

針對 Algorithm 2，我們的目標是刪除多餘的標籤，但是仍須確保任兩點可以在最大標籤 a 的限制下抵達對方。我們目前預計的嘗試方向如下：

1. **指數間隔標籤策略**：針對 Algorithm 2 的標籤刪除，我們初步考慮保留呈現指數增長的標籤集合。對於子圖 H 中的每一條邊 e ，僅保留 2 的冪次時間點 $\lambda'(e) = \{2^k | 2^k \leq a, k \in \mathbb{N}\}$ 。然而，此方法存在根本性的缺陷：由於時變路徑的時間標籤必須嚴格

遞增，若所有邊皆採用相同的標籤集合，則可行路徑的跳數 (hop count) 將被限制在 $O(\log a)$ 以內。此結果與 DCSS 演算法允許子圖直徑達 a 的特性相衝突，導致長距離傳輸失敗。為此，我們計畫針對每個邊 e ，引入偏移 (offset) δ_e 。修正後的標籤集合為 $\lambda'(e) = \{\delta_e + 2^k | \delta_e + 2^k \leq a, k \in \mathbb{N}\}$ 。我們預期透過適當設計 δ_e (例如利用 graph coloring 或是隨機演算法)，能錯開相鄰邊的可用時間區間，進而達成長距離傳輸。此方向之目標是將每條邊的標籤數由 a 降低至 $\text{polylog}(a)$ 。

2. **週期性標籤策略**：此策略類比於大眾運輸系統的定時發車機制。我們擬設定一週期 P (例如 $P = \sqrt{a}$ 或與直徑相關之參數)，對於邊 e 僅保留 P 的整數倍標籤 $\lambda'(e) = \{Pk | k \in \mathbb{N}\}$ 。很明顯地，此方法將 Algorithm 2 的標籤總數降低 P 倍。與前述策略相同，我們也會考慮引入偏移 δ_e ，達成長距離傳輸。
3. **基於有效時間窗口的標籤策略**：此策略計算每條邊的有效使用時段，進而刪除不在該時段內的冗餘標籤。具體執行邏輯如下：對於 Algorithm 2 第 2 行得到的 H 中的每個邊 e ，我們首先找出有哪些起點終點對 (p, q) 會經過邊 e 。我們針對 (p, q) 計算 e 的最早抵達時間， $EAT_{p,q}(e)$ ，即為 p 到 e 的最短距離，與 e 的最晚可離開時間， $LDT_{p,q}(e)$ ，即為 a 減去 e 至 q 的最短距離。令 $D(e) = \{(p, q) | p, q \in V \text{ 且 } p, q \text{ 在 } H \text{ 的最短路徑會經過 } e\}$ ，

$$EAT(e) = \min_{(p,q) \in D(e)} EAT_{p,q}(e), LDT(e) = \max_{(p,q) \in D(e)} LDT_{p,q}(e)。$$

對於邊 e ，我們僅保留位於區間 $[EAT(e), LDT(e)]$ 內的標籤。雖然此方法在直覺上能刪除顯而易見的冗餘標籤，但要建立其近似比仍具挑戰性。為了分析此策略的近似比，我們必須找到一個對應的 MAL 問題最佳解下界。我們將致力於探討如何利用 H 的結構特性來推導此下界，以分析保留的標籤數量與最佳解的差距倍數。另一方面，此方法也可以適用於 Steiner Minimum Aged Labelling (Steiner MAL) 問題。此問題為 MAL 問題的推廣，給定一 Terminal 集合 $U \subseteq V$ ，目標是尋找最少的標籤總數，使得 U 中的任兩點 $u, v \in U$ 皆存在時變路徑互相抵達。當 $U = V$ 時，此問題即退化為標準的 MAL 問題。對於 Steiner MAL 問題，我們可以減少 $D(e)$ 的大小，進而降低標籤數。

接著我們討論如何改善 Algorithm 4 在第二階段無法有效共用標籤的問題。Algorithm 4 利用 Dominating Set S 作為中繼骨幹，為了讓 S 中任兩點都可以抵達對方，本質上需要解決 Steiner MAL 問題，其中 Terminal 集合為 S 。Algorithm 4 直接建立 S 中任兩點的最短路徑，因此標籤無法有效共用。本計畫擬透過遞迴式階層架構解決 Steiner MAL 問題。因為 MAL 問題為 Steiner MAL 問題特例，所以解決 Steiner MAL 問題即可解決 MAL 問題。

我們的 Steiner MAL 演算法可視為 Algorithm 4 的遞迴版本。我們首先定義 U -Restricted d -Dominating Set：給定節點子集合 $U \subseteq V$ ，若某個集合 S 滿足 $S \subseteq U$ ，且所有 $u \in U$ 皆滿足 $d_G(u, S) \leq d$ ，則稱 S 為一個 U -Restricted d -Dominating Set。我們的演算法框架如下：令 $U_0 = V$ 為初始輸入。我們建立一系列的巢狀集合 $V = U_0 \supseteq U_1 \supseteq U_2 \supseteq \dots \supseteq U_k$ ，其中 U_{i+1} 為 U_i -Restricted d_{i+1} -Dominating Set。整體標籤指派呈現階層式架構，分為三個階段。

1. 第一階段：對於每一層 i ($0 \leq i < k$)，我們利用 Shortest Path Forest 建立由 U_i 抵達 U_{i+1} 的時變路徑。
2. 第二階段：此階段的目標為讓 U_{i+1} 內的任兩節點存在時變路徑。注意到此問題為一個 Steiner MAL 問題，其 Terminal 集合為 U_{i+1} 。與 Algorithm 4 不同，我們會透過遞迴的方式解決此 Steiner MAL 子問題。
3. 第三階段：利用 SPF 的逆向路徑，建立由 U_{i+1} 返回 U_i 的時變路徑。

此遞迴架構的關鍵在於 $d_1, d_2, \dots, d_k$ 的選擇。這是一個在深度與每層最大標籤之間的權衡問題。我們將致力於尋找一組最佳的序列 $d_1, d_2, \dots, d_k$ ，在滿足時間 a 的限制下，最小化總標籤數。

(五) 第二年研究方法：考慮 MAL-S 問題與 MAL-C 問題

A. MAL-S 問題演算法

MAL-S 問題為 MAL 問題之推廣，引入了額外的參數 α ，代表時變路徑長度相對於圖上最短路徑的最大容許拉伸程度 (stretch)。回顧 Algorithm 1，其主要透過建構單一生成樹 (spanning tree) T 來規劃時變路徑。然而，作為 MAL-S 問題的演算法，Algorithm 1 存在結構性的缺陷：單一樹結構可能嚴重破壞原圖的鄰近性，導致某些在底圖 G 中相鄰的節點對 (例如圖 3 的 a 點與 b 點)，在樹上的距離顯著增加，無法滿足拉伸限制 α 。

為了克服此樹狀結構限制，我們計劃採用 Subtree Cover，即使用多棵樹而非單一樹來覆蓋底圖。更明確地說，我們計劃尋找一個 (α, k) -Subtree Cover，其為一個樹的集合 $\mathcal{T} = \{T_1, T_2, \dots, T_k\}$ ，並滿足以下條件：

- **子圖性質**：每個 T_i 皆為底圖 G 的子圖，且為樹結構 (不必然為生成樹)。
- **有限拉伸**：對於任何節點對 $u, v \in V$ ， $\mathcal{T}$ 中至少存在一棵樹 T_i ，使得該樹上的距離滿足 $d_{T_i}(u, v) \leq \alpha \cdot d_G(u, v)$ 。

針對 Subtree Cover 的建構，文獻上已有成熟的演算法。對於任何參數 $p \geq 1$ ，Abraham 等人提出的演算法可計算出 $(p \log \log n, pn^{1/p})$ -Subtree Cover [ACE⁺20]。若考慮到底圖為平面圖 (planar graphs，常見於路網運輸圖)，則可利用 Gupta 等人的演算法計算出 $(3, O(\log n))$ -Subtree Cover [GKR05]。本計畫擬利用上述演算法作為前處理步驟，以獲得性質良好的 Subtree Cover。

然而，單純將 Algorithm 1 套用至 Subtree Cover 中的每一棵樹 T_i 仍不足以解決 MAL-S 問題。這是因為 Subtree Cover 僅保證了靜態距離的近似，而 Algorithm 1 的標籤指派策略 (強制經由根節點路由) 可能引入額外的時變延遲。舉例來說，圖 3 中的 x 與 y 雖然在樹 T 上距離很短，但因在中繼點有很長的等待時間，導致時變距離長達 $17 + 1 - 2$ 。

為解決此問題，我們擬優化樹上的路由策略：讓起點 x 僅需上行至 x 與 y 的最低共同祖先 (Lowest Common Ancestor, LCA) w ，即可轉折下行至終點 y ，無須繞行至 T_i 的根節點。直覺上，此策略等同於要求樹上任意子樹皆須具備時變連通性 (即對於 T_i 中任意以 v 為根的子樹，皆需能獨立運作)。ⁱⁱⁱ 若直接對所有子樹重複套用 Algorithm 1，將導致標籤數量劇增，造成極大的資源浪費。

因此，本階段的核心挑戰在於設計高效的標籤共用機制。為了減少標籤數，我們提出一種由下而上 (bottom-up) 的收縮策略，而非直接將 Algorithm 1 套用至 T_i 。具體執行步驟如下：我們從 T_i 中最深層的子樹開始，逐一將 Algorithm 1 的標籤邏輯應用於該子樹，以確保局部的時變連通與拉伸限制。一旦處理完畢，即將該子樹收縮 (contraction) 為一個單一的超節點 (super node)，接著再往上一層考慮包含此超節點的父子樹。透過此迭代過程，我們可以確保每一層級的局部拉伸程度皆受控，最終再透過全域的標籤調整串接各層路徑。此外，我們亦將探討跨樹標籤共用的可能性，即分析是否能讓 $\mathcal{T}$ 中結構相似或互不干擾的樹共用同一組時間標籤，以進一步降低總成本。

除了演算法設計，近似比分析亦為關鍵。雖然標準 MAL 問題的最佳解下界 (如 $n-1$) 亦適用於 MAL-S，但此下界對於具備拉伸限制的問題而言可能過於寬鬆，難以導出緊湊 (Tight) 的近似比。我們預計先從簡化設定 (無最大標籤限制、無向圖、無邊權重) 著手，致力於推導針對 MAL-S 問題更緊湊的下界，最終得到本計畫提出演算法的近似比。

另一方面，關於近似比下界 (inapproximability)，當 α 足夠大時。我們初步會直接延伸 MAL 問題的結果至 MAL-S 問題。當 α 較小時，我們也可嘗試沿用 Spanner 問題的近似比下

ⁱⁱⁱ 更明確地說，我們只需考慮至少有兩個子節點的節點 v 為根的子樹。

界至 MAL-S 問題。最後，我們過去曾透過 Min-Rep 問題推導其他具拉伸限制的網路設計問題的近似比下界 [Kuo19]，我們也會嘗試複製過去成功經驗，透過 Min-Rep 問題求得 MAL-S 問題的近似比下界。

B. MAL-C 問題演算法

最後，我們探討 MAL-C 問題的演算法設計。我們的核心策略是利用時間擴展圖 (Time-Expanded Graph, TEG)，將時變圖上的標籤決策問題轉換為靜態圖上的網路流設計問題。TEG 的建構方法主要參考 Akrida 等人的研究 [ACG⁺19]。為了簡化討論並釐清轉換機制，我們先作以下兩個簡化假設：1) 每個邊的行駛時間皆為單位時間 (即 $w(e) = 1$)。2) 已知每個邊的標籤集合 (對於 Akrida 等人的研究，邊標籤集合確實為輸入)。

TEG $G_T = (V_T, E_T)$ 的具體建構步驟如下 [ACG⁺19]：

1. **節點擴展**：針對底圖中的每一個節點 $v \in V$ ，我們在 G_T 中複製 $a + 1$ 份，分別記為 $v_1, v_2, \dots, v_{a+1}$ 。這些節點代表原節點 v 在不同時間點 $t \in \{1, 2, \dots, a + 1\}$ 的狀態。
2. **建立傳輸邊**：對於底圖中的每一條邊 $e = (u, v) \in E$ ，若其已知標籤集合 $\lambda(e)$ 包含時間點 t ，則我們在 G_T 中建立一條有向邊 (u_t, v_{t+1}) 。此邊代表在時間 t 透過邊 e 進行傳輸，並於 $t + 1$ 到達 v 。此邊的容量設定與底圖邊 e 的容量相同。
3. **建立暫存邊**：對於底圖中的每一個節點 $v \in V$ ，我們在 G_T 中依序連接其不同時間點的副本，建立邊 (v_t, v_{t+1}) ，其中 $t \in \{1, 2, \dots, a\}$ 。這些邊代表貨物選擇留在節點 v 等待下一個時間點 (暫存)。依據假設，各節點暫存空間無限，故這些等待邊的容量設為無限大。

不難看出上述建構方式可以延伸至每個邊有不同行駛時間，甚至是節點暫存容量有限的情形。

圖 7 展示了一個簡單的範例。圖中紅色數字代表邊的容量限制。在此例中，最大標籤為 4，意味著我們考慮的時間區間至時刻 5 為止。透過 G_T ，原本時變圖上的動態路徑被轉化為靜態圖上的一般路徑。

針對本計畫探討的 MAL-C 問題，由於邊的標籤集合 $\lambda(e)$ 並非輸入資料，而是待決策的變數，我們採取完全擴展的策略：假設底圖中每一條邊 $e = (u, v)$ 在所有可能的時間點 $t \in \{1, \dots, a\}$ 皆存在對應的傳輸邊 (u_t, v_{t+1}) 。換言之，我們建構一個包含所有潛在發車可能性的 G_T 。另一方面，MAL-C 問題的底圖 G 沒有邊容量上限，但是有傳輸需求。在此設定下，MAL-C 問題轉化為在 G_T 上尋找一組流量分佈，以最小化每個邊流量的無條件進位總和。在數學結構上，這是一個在有向圖上的 Network Loading Problem，該問題的計算複雜度取決於需求量 (demands) 的性質：

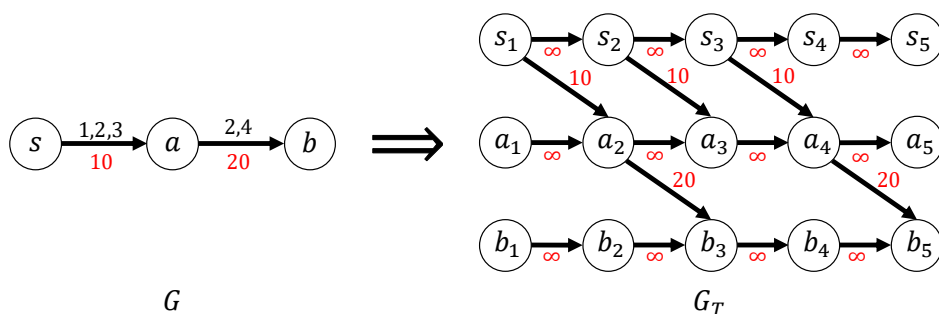


圖 7: 底圖 G 與其對應的時間擴展圖 G_T 。左圖為具已知標籤的時變圖，右圖為展開後的靜態圖。紅色數字表示容量限制， G_T 中水平的邊為暫存邊，斜向的邊為傳輸邊。

1. **整數需求情形**：若所有貨物需求量 $\text{dem}(d_i)$ 皆為整數，此問題可直接透過最小成本流問題（minimum cost flow）求得最佳解。換句話說，在此狀況下，我們可以得到精確演算法（exact algorithm）。
2. **小數需求情形**：若需求量包含非整數，此問題屬於 NP-hard。更具體而言，當需求量極小（ $\text{dem}(d_i)$ 非常靠近 0）時，此問題包含 Directed Steiner Tree 問題作為特例，已知其近似比下界為 $\Omega(k^\epsilon)$ [DK99]。儘管此問題極具挑戰性，但我們可以直接使用有向圖上的 Buy-at-Bulk Network Design 演算法。目前已知最佳的近似比為 $O(\min(h^{1/2+\epsilon}, n^{4/5+\epsilon}))$ [Ant10]。

當在 G_T 上求解完成後，我們可以由流量解回推底圖的標籤設定：若 G_T 上的傳輸邊 (u_t, v_{t+1}) 承載了正流量 $f > 0$ ，則隱含底圖邊 (u, v) 必須包含標籤 t ，且該時刻需購買的容量（發車數）為 $\lceil f \rceil$ 。

上述流程雖在理論上可行，但面臨一個計算上的瓶頸：TEG G_T 的大小（節點與邊數）與最大標籤限制 a 的數值呈線性關係。這導致演算法的時間複雜度為偽多項式時間（pseudo-polynomial time），當 a 很大時將難以計算。為此，我們的目標是優化 G_T 的建構方式。Hoppe 與 Tardos 在 Quickest Transshipment Problem 中也解決了將偽多項式時間降低為多項式時間的問題 [HT00]。我們計畫將他們的方法應用至 MAL-C 問題，設計多項式時間的近似演算法或精確演算法。

四、 預期完成之工作項目及成果

在第三節中，我們探討了 MAL 問題的過去研究結果。目前文獻上最主要的成果來自 Carnevale 等人 [CDO25]，他們針對無向圖建立了一系列近似比的上下界。然而，現有的演算法存在顯著的限制：首先，它們僅關注連通性與標籤數量，忽略了路徑的拉伸程度，這可能導致貨物運輸產生極大的延遲；其次，它們未考慮邊的容量限制，這與實際物流網路的需求不符。此外，針對最大標籤限制 a 介於 $\text{Diam}(G)$ 與 $2 \text{Diam}(G)$ 之間的關鍵區間，現有的近似演算法仍有改善空間。鑑於上述研究缺口，本計畫旨在設計具拉伸限制（MAL-S）與容量限制（MAL-C）的高效近似演算法，並優化現有 MAL 問題的近似比。

為達成此目標，我們將整合多種演算法技術。針對 MAL-S 問題，我們計畫採用 Subtree Cover 技術 [ACE⁺20, GKR05] 取代單一生成樹，並使用 Bottom-Up Contraction 策略以減少標籤。針對 MAL-C 問題，我們的核心策略是參考 Akrida 等人 [ACG⁺19] 的方法，將時變圖轉換為時間擴展圖 TEG，將問題轉化為靜態網路流問題。為了克服 TEG 大小隨最大標籤限制 a 呈線性成長導致的偽多項式時間複雜度問題，我們將借鏡 Hoppe 與 Tardos 在 Quickest Transshipment Problem 中的方法 [HT00] 來優化 G_T 的大小。

在第一年，我們計畫改良現有的 MAL 演算法，特別是針對 $a < 1.5 \text{Diam}(G)$ 的情形與有向圖的設定，並針對基於 DCSS 與 Dominating Set 的演算法框架（Algorithm 2 與 Algorithm 4）進行標籤優化。在第二年，我們將正式處理 MAL-S 與 MAL-C 問題，設計演算法並分析其近似比。我們目前已針對 MAL 問題進行初步研究，取得了以下簡單成果：

1. **針對關鍵區間 $a < 1.5 \text{Diam}(G)$ 的優化**：令 $a = (1+c) \text{Diam}(G)$ ，我們推導出 $O(\frac{1}{\sqrt{c}} n^{4/5+0.5\epsilon})$ -近似演算法。此結果證明了在 $\text{Diam}(G)$ 較大時，我們的演算法優於現有文獻（Theorem 6）。
2. **有向圖上的近似演算法**：針對有向圖且 $a \geq 2 \text{Diam}(G)$ 的情形，我們提出了 2-近似演算法。這是目前文獻上針對有向圖 MAL 問題所提出的首個常數近似演算法（Theorem 7）。

此計畫後續將基於上述初步成果，進一步擴展至更複雜的 MAL-S 與 MAL-C 問題。預期完成之工作項目及成果條列如下：

第一年

1. 針對標準 MAL 問題，設計適用於 $a < 1.5 \text{Diam}(G)$ 情形與有向圖的近似演算法。
2. 改良基於 DCSS 的演算法框架 (Algorithm 2)，利用指數間隔或週期性標籤策略刪除冗餘標籤，降低近似比。
3. 發展遞迴 Dominating Set 演算法，以優化 Algorithm 4 的標籤共用機制。
4. 預期成果：將上述演算法設計與分析結果發表至演算法領域的國際學術研討會。

第二年

1. 針對 MAL-S 問題，設計基於 Subtree Cover 與 Bottom-Up Contraction 的演算法，並推導其近似比。另一方面，將 MAL 問題、Spanner 問題、Min-Rep 問題的近似比下界延伸至 MAL-S 問題。
2. 針對 MAL-C 問題，利用時間擴展圖 (TEG G_T) 將其轉化為網路流問題，並應用 Hoppe 與 Tardos 的技術優化 TEG G_T 結構，設計多項式時間的近似演算法或精確演算法 (exact algorithm)。
3. 預期成果：將 MAL-S 與 MAL-C 的研究成果發表至演算法領域的國際學術研討會。

參考文獻

- [ABD⁺25] Guillaume Aubian, Filippo Brunelli, Feodor F Dragan, Guillaume Ducoffe, Michel Habib, Allen Ibiapina, and Laurent Viennot. On the complexity of computing a fastest temporal path in interval temporal graphs, 2025.
- [ABF⁺24] Sebastian Angrick, Ben Bals, Tobias Friedrich, Hans Gawendowicz, Niko Hastrich, Nicolas Klodt, Pascal Lenzner, Jonas Schmidt, George Skretas, and Armin Wells. How to Reduce Temporal Cliques to Find Sparse Spanners. In *ESA*, 2024.
- [ACE⁺20] Ittai Abraham, Shiri Chechik, Michael Elkin, Arnold Filtser, and Ofer Neiman. Ramsey spanning trees and their applications. *ACM Trans. Algorithms*, 16(2), March 2020.
- [ACG⁺19] Eleni C. Akrida, Jurek Czyzowicz, Leszek Gąsieniec, Łukasz Kuszner, and Paul G. Spirakis. Temporal flows in temporal networks. *Journal of Computer and System Sciences*, 103:46–60, 2019.
- [ACIM99] D. Aingworth, C. Chekuri, P. Indyk, and R. Motwani. Fast estimation of diameter and shortest paths (without matrix multiplication). *SIAM Journal on Computing*, 28(4):1167–1181, 1999.
- [Ant10] Spyridon Antonakopoulos. Approximating directed buy-at-bulk network design. In *WAOA*, 2010.
- [BDG⁺22] Davide Bilò, Gianlorenzo D’Angelo, Luciano Gualà, Stefano Leucci, and Mirko Rossi. Sparse Temporal Spanners with Low Stretch. In *ESA*, 2022.

- [BHNN20] Matthias Bentert, Anne-Sophie Himmel, André Nichterlein, and Rolf Niedermeier. Efficient computation of optimal temporal walks under waiting-time constraints. *Applied Network Science*, 5(1):73, 2020.
- [BKMP10] Surender Baswana, Telikepalli Kavitha, Kurt Mehlhorn, and Seth Pettie. Additive spanners and (α, β) -spanners. *ACM Trans. Algorithms*, 7(1), December 2010.
- [Bod22] Greg Bodwin. A note on distance-preserving graph sparsification. *Information Processing Letters*, 174:106205, 2022.
- [BPE25] Thomas Bellitto, Jules Bouton Popper, and Bruno Escoffier. Temporal Connectivity Augmentation. In *SAND*, 2025.
- [BWD⁺12] Benjamin Blonder, Tina W. Wey, Anna Dornhaus, Richard James, and Andrew Sih. Temporal dynamics and network analysis. *Methods in Ecology and Evolution*, 3(6):958–972, 2012.
- [CDKL20] Eden Chlamtáč, Michael Dinitz, Guy Kortsarz, and Bundit Laekhanukit. Approximating spanners and directed steiner forest: Upper and lower bounds. *ACM Trans. Algorithms*, 16(3), June 2020.
- [CDO25] Daniele Carnevale, Gianlorenzo D’Angelo, and Martin Olsen. Approximating optimal labelings for temporal connectivity. In *AAAI*, 2025.
- [CG20] Keerti Choudhary and Omer Gold. Extremal distances in directed graphs: tight spanners and near-optimal approximation algorithms. In *ACM-SIAM SODA*, 2020.
- [CPS21] Arnaud Casteigts, Joseph G. Peters, and Jason Schoeters. Temporal cliques admit sparse spanners. *Journal of Computer and System Sciences*, 121:1–17, 2021.
- [DK99] Yevgeniy Dodis and Sanjeev Khanna. Design networks with bounded pairwise distance. In *ACM STOC*, 1999.
- [DOS12] Frank Dehne, Masoud T Omran, and Jörg-Rüdiger Sack. Shortest paths in time-dependent fifo networks. *Algorithmica*, 62(1):416–435, 2012.
- [DPSW18] Julian Dibbelt, Thomas Pajor, Ben Strasser, and Dorothea Wagner. Connection scan algorithm. *ACM J. Exp. Algorithmics*, 23, October 2018.
- [EMW25] Thomas Erlebach, Nils Morawietz, and Petra Wolf. Parameterized algorithms for multi-label periodic temporal graph realization. *Theoretical Computer Science*, 1051:115412, 2025.
- [FF58] L. R. Ford and D. R. Fulkerson. Constructing maximal dynamic flows from static flows. *Operations Research*, 6(3):419–433, 1958.
- [GKR05] Anupam Gupta, Amit Kumar, and Rajeev Rastogi. Traveling with a pez dispenser (or, routing issues in mpls). *SIAM Journal on Computing*, 34(2):453–474, 2005.
- [HT00] Bruce Hoppe and Éva Tardos. The quickest transshipment problem. *Mathematics of Operations Research*, 25(1):36–62, 2000.
- [JS24] Anuj Jain and Sartaj Sahni. Shortest paths in interval temporal graphs. In *IC3*, IC3-2024, 2024.

- [KKGB10] Pablo Kaluza, Andrea Kölzsch, Michael T. Gastner, and Bernd Blasius. The complex network of global cargo ship movements. *Journal of The Royal Society Interface*, 7(48):1093–1103, 2010.
- [KKK02] David Kempe, Jon Kleinberg, and Amit Kumar. Connectivity and inference problems for temporal networks. *Journal of Computer and System Sciences*, 64(4):820–842, 2002.
- [KMMS24] Nina Klobas, George B. Mertzios, Hendrik Molter, and Paul G. Spirakis. The complexity of computing optimum labelings for temporal connectivity. *Journal of Computer and System Sciences*, 146:103564, 2024.
- [KMMS25] Nina Klobas, George B. Mertzios, Hendrik Molter, and Paul G. Spirakis. Temporal graph realization from fastest paths. *Theoretical Computer Science*, 1056:115508, 2025.
- [Kuo19] Tung-Wei Kuo. On the approximability and hardness of the minimum connected dominating set with routing cost constraint. *Theoretical Computer Science*, 793:140–151, 2019.
- [MM75] Amram Meir and John Moon. Relations between packing and covering numbers of a tree. *Pacific Journal of Mathematics*, 61(1):225–233, 1975.
- [MMMS25a] George Mertzios, Hendrik Molter, Nils Morawietz, and Paul Spirakis. Realizing temporal transportation trees. In *WG*, 2025.
- [MMMS25b] George Mertzios, Hendrik Molter, Nils Morawietz, and Paul Spirakis. Temporal graph realization with bounded stretch. In *MFCS*, volume 345, 2025.
- [MMS19] George B Mertzios, Othon Michail, and Paul G Spirakis. Temporal network optimization subject to connectivity constraints. *Algorithmica*, 81(4):1416–1449, 2019.
- [PD05] Mercedes Pascual and Jennifer A Dunne. *Ecological Networks: Linking Structure to Dynamics in Food Webs*. Oxford University Press, 12 2005.
- [PE14] Giovanni Petri and Paul Expert. Temporal stability of network partitions. *Phys. Rev. E*, 90:022813, Aug 2014.
- [RC13] Ursula Redmond and Pádraig Cunningham. Temporal subgraph isomorphism. In *IEEE/ACM ASONAM*, 2013.
- [SWP⁺14] Ingo Scholtes, Nicolas Wider, René Pfitzner, Antonios Garas, Claudio J Tessone, and Frank Schweitzer. Causality-driven slow-down and speed-up of diffusion in non-markovian temporal networks. *Nature communications*, 5(1):5024, 2014.
- [TMGV23] Bill Tao, Maleeha Masood, Indranil Gupta, and Deepak Vasisht. Transmitting, fast and slow: Scheduling satellite traffic through space and time. In *ACM Mobicom*, 2023.
- [WCK⁺16] Huanhuan Wu, James Cheng, Yiping Ke, Silu Huang, Yuzhen Huang, and Hejun Wu. Efficient algorithms for temporal path computation. *IEEE Transactions on Knowledge and Data Engineering*, 28(11):2927–2942, 2016.
- [WDdACG12] John Whitbeck, Marcelo Dias de Amorim, Vania Conan, and Jean-Loup Guillaume. Temporal reachability graphs. In *ACM Mobicom*, 2012.